

未命名1

Andata-AI 项目学习复盘总结（面向 Java 初学者）

本文是结合你之前所有提问整理的项目主流程与核心代码模块说明。

目标：以后回看项目时，能快速理解整个链路：

请求怎么进来 → 调 Kepler → 汇总返回

1. 项目整体定位：它到底是干什么的？

1.1 项目本质

Andata-AI 不是一个自己计算数据的系统，它本质是：

Kepler 智能研判系统的 API 转发 + 聚合服务

1.2 它做的事情

- 接收外部请求（手机号、WiFi、IP 等）
- 调用 Kepler 外部接口获取数据
- 做少量过滤、排序、条数限制
- 汇总成统一返回格式返回给前端

2. 核心请求链路（最重要主线）

完整调用链路如下：

```
用户请求 API
↓
Controller 接收请求
↓
KeplerApiTools（单接口）或 KeplerTools（全量汇总）
↓
performRequest() 发 HTTP 请求到 Kepler 外部系统
↓
解析 JSON → DTO 对象
↓
```

3. 项目配置入口：application.yml

文件位置：

```
src/main/resources/application.yml
```

核心内容：

```
server:
  port: 9981

tool:
  kepler:
    base-url: https://test-kepler.andataoa.com/api
```

作用：

- 项目启动端口：9981
- Kepler 外部接口统一前缀 base-url

最终请求形式：

```
base-url + uri
```

4. Controller 层：接口入口

4.1 CommonController（简化版入口）

路径：

```
src/main/java/.../CommonController.java
```

特点：

- 接口少 (models + analysis)
- 参数来自 URL RequestParam
- 更像 demo 或早期版本

核心调用：

```
return keplerTools.getCompleteAnalysisData(...)
```

4.2 KeplerApiController (正式 API 网关)

路径：

```
src/main/java/.../KeplerApiController.java
```

特点：

- 接口齐全，对应 Kepler API 文档每一项
- 参数使用 JSON Body (CommonRequest)
- 支持单接口查询 + 全量查询

例子：

```
return keplerApiTools.getDeviceInfo(...)
```

全量分析：

```
return keplerTools.getCompleteAnalysisData(...)
```

5. 请求体结构：CommonRequest

路径：

```
src/main/java/.../CommonRequest.java
```

它是所有接口统一输入 JSON 模板：

```
public class CommonRequest {  
    private String keyType;  
    private String key;  
    private String wifiMac;  
    private String ip;  
    private Integer limit;  
    private String startTime;  
    private String endTime;  
}
```

前端请求示例：

```
{  
  "keyType": "phone",  
  "key": "13669580956",  
  "limit": 10  
}
```

6. 两种调用模式（核心区别）

6.1 单接口调用：KeplerApiTools

定位：

一个方法对应一个 Kepler 外部接口。

特点：

- 快
- 只查某一项数据

例子：

- getBaseInfo()
- getDeviceInfo()
- getHistoryWifi()
- getRecentIpDetail()

适合：

前端按需查询某个模块。

6.2 全量接口调用：KeplerTools

定位：

一次调用多个接口 → 汇总成综合报告。

特点：

- CompletableFuture 并发调用多个数据源
- 汇总 builder 返回完整分析结果

核心方法：

```
getCompleteAnalysisData()
```

7. KeplerTools 主流程（全量分析核心）

7.1 并发调用多个接口

```
CompletableFuture<BaseInfoDTO> baseInfoFuture = supplyAsync(...)
CompletableFuture<DeviceInfoDTO> deviceInfoFuture = supplyAsync(...)
CompletableFuture<HistoryIPDTO> historyIPFuture = supplyAsync(...)
```

每个 Future：

- 调用 performRequest()
- 解析 JSON
- 成功返回 DTO
- 失败记录 errorResults

7.2 等待全部任务完成

```
CompletableFuture.allOf(...).join();
```

意思：

等待所有接口都查完再继续。

7.3 汇总结果返回

```
CompleteAnalysisData2DTO.builder()  
    .baseInfo(...)  
    .deviceInfo(...)  
    .historyIP(...)  
    ...  
    .build();
```

最终：

```
return Result.ok(aiString);
```

8. HTTP 请求封装：performRequest()

位置：

```
KeplerTools.performRequest()
```

作用：

所有外部接口调用统一入口。

核心逻辑：

```
HttpRequest.post(url)  
    .header("Authorization", authorization)  
    .header("userSession", userSession)  
    .body(JSON.toJSONString(requestBody))  
    .execute()  
    .body();
```

9. 返回结构统一封装：Result

路径：

```
src/main/java/.../Result.java
```

统一返回 JSON:

```
{
  "code": 200,
  "message": "操作成功",
  "data": ...
}
```

常用:

```
Result.ok(data)
Result.error(code, message)
```

10. 测试代码: AndataAiApplicationTests

路径:

```
src/test/java/.../AndataAiApplicationTests.java
```

作用:

- 本地快速验证 Kepler 接口是否可用
- RestClient 发请求并解析 DTO

类似 Python requests 测试脚本。

11. 推荐复盘学习顺序

第一阶段: 主线跑通

1. application.yml (base-url)
2. KeplerApiController (接口入口)
3. CommonRequest (输入参数)
4. Result (返回格式)

第二阶段：单接口理解

- 5. KeplerApiTools（一个接口一个方法）
- 6. performRequest（外部请求怎么发）

第三阶段：全量聚合理解

- 7. KeplerTools（并发汇总）
 - 8. CompleteAnalysisData2DTO（最终综合报告结构）
-

12. 项目一句话总结

Andata-AI 是一个 Kepler 智能研判 API 聚合服务：

Controller 接收请求 → Tools 调 Kepler 外部接口 → 汇总结果 → Result 统一返回。